

Atividade de Comitês de Classificadores

IMD1101 - Aprendizado de Máquina Supervisionado

Discente: Clara Virgínia Nascimento Santos
Data: 31/05/2026

Objetivo: Avaliar quatro técnicas de comitês de classificadores (Bagging, Random Forest, Stacking e Voting) em 12 bases de dados de imagens de cães e gatos, selecionadas a partir da atividade anterior de k-NN.

1. BAGGING

Estimadores: kNN (k=4), Decision Tree (depth=4), Naive Bayes, MLP (100,) | n_estimators: 10, 15, 20

	Base	kNN(10)	kNN(15)	kNN(20)	DT(10)	DT(15)	DT(20)	NB(10)	NB(15)	NB(20)	MLP(10)	MLP(15)	MLP(20)
PCA_075_HOG_256_32x32 (86 atrib.)	0.61694	0.688	0.679	0.683	0.688	0.694	0.676	0.674	0.674	0.644	0.658	0.673	
PCA_075_HOG_128_16x16 (105 atrib.)	0.61675	0.668	0.669	0.621	0.646	0.642	0.678	0.671	0.682	0.663	0.656	0.656	
PCA_090_HOG_256_32x32 (175 atrib.)	0.61664	0.661	0.657	0.658	0.665	0.674	0.662	0.662	0.661	0.638	0.642	0.653	
HOG_128_32x32 (324 atrib.)	0.65	0.647	0.651	0.644	0.651	0.648	0.674	0.673	0.675	0.571	0.558	0.58	
LBP_256_12R (98 atrib.)	0.612	0.627	0.63	0.656	0.668	0.665	0.577	0.589	0.589	0.463	0.463	0.463	
HOG_256_32x32 (1764 atrib.)	0.651	0.655	0.637	0.685	0.684	0.678	0.686	0.689	0.692	0.679	0.679	0.685	
PCA_090_HOG_128_16x16 (210 atrib.)	0.61649	0.641	0.656	0.629	0.653	0.67	0.675	0.672	0.667	0.672	0.635	0.635	
LBP_256_3R (26 atrib.)	0.639	0.639	0.647	0.645	0.651	0.666	0.615	0.615	0.608	0.463	0.463	0.463	
PCA_075_HOG_256_16x16 (199 atrib.)	0.61642	0.646	0.641	0.663	0.649	0.662	0.623	0.618	0.624	0.67	0.696	0.696	
HOG_128_16x16 (1764 atrib.)	0.635	0.63	0.64	0.64	0.662	0.653	0.7	0.7	0.706	0.691	0.688	0.687	
LBP_256_6R (50 atrib.)	0.632	0.624	0.611	0.651	0.673	0.666	0.635	0.626	0.63	0.463	0.463	0.463	
PCA_075_HOG_256_8x8 (320 atrib.)	0.61603	0.618	0.63	0.665	0.664	0.662	0.603	0.597	0.598	0.691	0.666	0.672	

2. RANDOM FOREST

criterion: gini, entropy | n_estimators: 10, 20, 30, 100

	Base	gini(10)	gini(20)	gini(30)	gini(100)	ent(10)	ent(20)	ent(30)	ent(100)
PCA_075_HOG_256_32x32 (86 atrib.)	0.639	0.663	0.664	0.656	0.625	0.683	0.684	0.654	
PCA_075_HOG_128_16x16 (105 atrib.)	0.612	0.652	0.636	0.655	0.633	0.628	0.612	0.62	
PCA_090_HOG_256_32x32 (175 atrib.)	0.64	0.643	0.64	0.641	0.604	0.629	0.622	0.626	
HOG_128_32x32 (324 atrib.)	0.659	0.688	0.685	0.677	0.645	0.677	0.647	0.681	
LBP_256_12R (98 atrib.)	0.673	0.675	0.676	0.682	0.632	0.666	0.66	0.681	
HOG_256_32x32 (1764 atrib.)	0.662	0.692	0.709	0.719	0.687	0.703	0.715	0.703	
PCA_090_HOG_128_16x16 (210 atrib.)	0.606	0.608	0.595	0.583	0.596	0.617	0.614	0.584	
LBP_256_3R (26 atrib.)	0.653	0.661	0.657	0.682	0.65	0.678	0.684	0.686	
PCA_075_HOG_256_16x16 (199 atrib.)	0.626	0.682	0.676	0.637	0.637	0.655	0.653	0.625	
HOG_128_16x16 (1764 atrib.)	0.661	0.672	0.685	0.697	0.683	0.683	0.691	0.692	
LBP_256_6R (50 atrib.)	0.652	0.639	0.661	0.699	0.684	0.673	0.683	0.69	
PCA_075_HOG_256_8x8 (320 atrib.)	0.572	0.617	0.636	0.609	0.597	0.613	0.63	0.597	

3. STACKING

Pool heterogêneo (MLP, KNN, DT, NB) | Tamanhos: 5, 10, 15, 20 classificadores | Meta: LogisticRegression

	Base	Clf(5)	Clf(10)	Clf(15)	Clf(20)
PCA_075_HOG_256_32x32 (86 atrib.)		0.703	0.725	0.735	0.734
PCA_075_HOG_128_16x16 (105 atrib.)		0.665	0.692	0.678	0.713
PCA_090_HOG_256_32x32 (175 atrib.)		0.666	0.693	0.706	0.716
	HOG_128_32x32 (324 atrib.)	0.648	0.656	0.68	0.68
	LBP_256_12R (98 atrib.)	0.614	0.634	0.611	0.611
	HOG_256_32x32 (1764 atrib.)	0.649	0.666	0.699	0.717
PCA_090_HOG_128_16x16 (210 atrib.)		0.636	0.661	0.674	0.704
	LBP_256_3R (26 atrib.)	0.608	0.634	0.625	0.621
PCA_075_HOG_256_16x16 (199 atrib.)		0.634	0.666	0.66	0.697
	HOG_128_16x16 (1764 atrib.)	0.618	0.656	0.688	0.706
	LBP_256_6R (50 atrib.)	0.591	0.65	0.635	0.635
PCA_075_HOG_256_8x8 (320 atrib.)		0.616	0.637	0.634	0.695

4. VOTING

Pool heterogêneo (MLP, KNN, DT, NB) | Tamanhos: 5, 10, 15, 20 classificadores | voting='soft'

	Base	Clf(5)	Clf(10)	Clf(15)	Clf(20)
PCA_075_HOG_256_32x32 (86 atrib.)		0.675	0.716	0.709	0.713
PCA_075_HOG_128_16x16 (105 atrib.)		0.654	0.686	0.675	0.688
PCA_090_HOG_256_32x32 (175 atrib.)		0.623	0.675	0.694	0.684
HOG_128_32x32 (324 atrib.)		0.641	0.662	0.698	0.702

	LBP_256_12R (98 atrib.)	0.662	0.65	0.636	0.63
	HOG_256_32x32 (1764 atrib.)	0.628	0.664	0.708	0.712
PCA_090	HOG_128_16x16 (210 atrib.)	0.62	0.65	0.664	0.674
	LBP_256_3R (26 atrib.)	0.648	0.649	0.662	0.664
PCA_075	HOG_256_16x16 (199 atrib.)	0.629	0.649	0.663	0.681
	HOG_128_16x16 (1764 atrib.)	0.625	0.655	0.707	0.71
	LBP_256_6R (50 atrib.)	0.61	0.652	0.661	0.646
PCA_075	HOG_256_8x8 (320 atrib.)	0.596	0.617	0.645	0.66

5. ANÁLISE ESTATÍSTICA (Teste de Friedman)

Bagging: Teste de Friedman entre kNN, DT, NB e MLP (com `n_estimators` 10, 15, 20)

Random Forest: Teste de Friedman entre gini e entropy (com `n_estimators` 10, 20, 30, 100)

Stacking: Teste de Friedman entre tamanhos de pool (5, 10, 15, 20 classificadores)

Voting: Teste de Friedman entre tamanhos de pool (5, 10, 15, 20 classificadores)

Comparação Final (melhores modelos selecionados):

Estatística de Friedman = 4.5000 | p-valor = 0.2123

Resultado: Sem diferença estatística significativa ($p > 0.05$) entre os melhores modelos.

Critério de desempate: Maior F1-score médio (10-fold CV)

6. CONCLUSÃO E RECOMENDAÇÃO

Melhores modelos selecionados por técnica:

- **Bagging:** DT com `n_estimators=20` (F1 médio CV = 0.6651)
- **Random Forest:** gini com `n_estimators=100` (F1 médio CV = 0.6614)
- **Stacking:** Pool de 20 classificadores (F1 médio CV = 0.6857) ✓ **MELHOR**
- **Voting:** Pool de 20 classificadores (F1 médio CV = 0.6803)

Recomendação Final:

O modelo **Stacking com 20 classificadores** apresentou o melhor desempenho geral com F1-score médio de **0.6857**. Apesar de não haver diferença estatisticamente significativa entre os melhores modelos ($p = 0.2123$), o Stacking se destaca por:

1. Maior F1-score médio entre as 12 bases de dados
2. Melhor aproveitamento da heterogeneidade de classificadores
3. Desempenho mais consistente no Holdout (70/30)
4. Capacidade de combinar diferentes tipos de modelos (MLP, kNN, DT, NB) de forma equilibrada

Observação: Embora o Voting também tenha resultado muito próximo (F1 = 0.6803), o Stacking com meta-learner (LogisticRegression) conseguiu aproveitar melhor as previsões dos classificadores base.